Policy delivery
Component 4 → Component 3

A shared integration contract for deadline, Leaky Bucket and retry-budget updates.

Component 4
Calculate and send policy

→

Component 3
Receive and distribute

→

Sidecars
Apply and acknowledge

1 Receiving endpoints

Component 3 hosts these endpoints and supplies the reachable host and port.

```
POST /api/v1/policies/deadlines
```

```
POST /api/v1/policies/leaky-bucket
```

```
POST /api/v1/policies/retry-budget
```

Base URL: `http://<COMPONENT3_HOST>:<PORT>`

POLICY	ACTION	TARGET / UNIT
Deadline	`CHAIN_DEADLINE`	Method + normalized route `milliseconds`
Leaky Bucket	`LEAKY_BUCKET`	Service name `requests/second`
Retry budget	`RETRY_BUDGET`	Service name `extra attempts/original request`

2 Request headers

```
Content-Type: application/json
Accept: application/json
Idempotency-Key: <proposal.id>
```

`Authorization: Bearer <agreed-token>` is optional. Component 4 omits it when no receiver token is configured.

> **Approval rule:** deadlines are sent only after human approval. Leaky Bucket and retry-budget updates are automatic when required evidence and safety checks pass.

Shared JSON format

All three endpoints receive a `PolicyChange` message with a nested `proposal`. This example shows an approved deadline update.


```
{
  "proposal": {
    "id": "591fce21-d942-41b5-a24d-821755cc3c62",
    "target": "POST /api/orders",
    "action": "CHAIN_DEADLINE",
    "source": "TEMPO",
    "baseVersion": "v1",
    "currentValue": 1000,
    "proposedValue": 1440,
    "unit": "milliseconds",
    "generatedAt": "2026-10-15T10:00:00Z",
    "expiresAt": "2026-10-15T10:05:00Z",
    "measurements": {
      "baselineP99Ms": 1000,
      "recentP99Ms": 1200,
      "failureRate": 0.02
    },
    "evidence": ["Persistent deadline failures supported this change"]
  },
  "decision": "APPROVED",
  "reviewer": "sarala",
  "decidedAt": "2026-10-15T10:01:00Z",
  "delivery": "READY",
  "detail": "Awaiting delivery and enforcement acknowledgement",
  "attempts": 0,
  "lastAttempt": null,
  "appliedVersion": null
}
```


Illustrative values only. Every real message uses its own UUID and fresh UTC timestamps. Nullable metadata must be accepted. Measurements vary by action; evidence text is explanatory.

Values for the other two endpoints

FIELD	LEAKY BUCKET EXAMPLE	RETRY-BUDGET EXAMPLE
`target`	`order`	`order`
`action`	`LEAKY_BUCKET`	`RETRY_BUDGET`
`source`	`PROMETHEUS`	`PROMETHEUS`
Current → proposed	`120 → 90`	`0.05 → 0.02`
`decision`	`AUTOMATIC`	`AUTOMATIC`

Automatic reviewer: `adaptive-controller`. Use the units on page 1. Leaky Bucket changes the rate, not bucket capacity. Retry ratio `0.02` means two extra attempts per 100 original requests, not two retries per request.

Versioning and delivery

3 Policy version rules

Component 4 sends `baseVersion`: the current policy reference used for calculation. **Component 3 returns `policyVersion`:** a new version after confirmed application.

Current `v1` → recommendation uses `v1` → apply → `v2`

Maintain versions consistently per target. The next recommendation uses the confirmed version. Initial deadline references may be `prometheus-config:1000`; verify that startup configuration instead of assuming every first version is `v1`.

The URL segment `/api/v1/` versions the API contract. It is unrelated to applied policy version strings.

4 Acceptance response from Component 3

After validation, return **HTTP 202** with:

```
{"status": "ACCEPTED"}
```

Component 4 records `SENT`. Acceptance does not mean the sidecars have applied the policy.

5 Confirm application to Component 4

After the intended sidecars confirm application, Component 3 calls:

```
POST http://<COMPONENT4_HOST>:8090/api/v2/intelligence/changes/{proposalId}/acknowledge
Content-Type: application/json
X-RRPE-Control-Token: <Component4-control-token>
```

```
{
  "status": "APPLIED",
  "policyVersion": "v2",
  "appliedValue": 1440
}
```

Use the original proposal ID. The value must exactly match `proposedValue`; the version must be non-empty and different from `baseVersion`. The callback returns **HTTP 204** on success. Its control token is required independently of optional receiver bearer authentication.

If application is already confirmed immediately, Component 3 may return the APPLIED JSON directly to the original POST. Confirm promptly while the proposal and current reference remain valid.

6 Receiver checks

- Deduplicate by proposal ID; validate action, target, units, decision and expiry.
- Check current value and base version before applying. Reject stale updates.
- Reject `MOCK` messages. Return non-2xx for invalid data or conflicts, such as 400 or 409.
- Do not report `APPLIED` for partial sidecar application.